SnapPark Dissertation

UNDERGRADUATE INDIVIDUAL PROJECT
SnapPark: An Intelligent Microservices-Based Parking Violation Detection System
- Abstract
- Declaration
- Acknowledgments
- Contents
- List of Figures
- List of Tables
- Listings

Chapter 1
Introduction
- 1.1 Aim of the project
- 1.2 Project objectives
- 1.3 Report structure

Chapter 2
Literature Review
- 2.1 Architectural Paradigms
  - 2.1.1 Monolithic Architecture
  - 2.1.2 Service-Oriented Architecture (SOA)
  - 2.1.3 Microservices Architecture
  - 2.1.4 Choice of Architecture
- 2.2 Cloud Computing and the Cloud Continuum
- 2.3 Addressing Challenges in Microservice-Based Designs
  - 2.3.1 Microservice Identification and Domain-Driven Design
  - 2.3.2 Data Management
  - 2.3.3 API Gateway
  - 2.3.4 Service Discovery
  - 2.3.5 Communication: Synchronous vs Asynchronous
  - 2.3.6 Containerisation and Orchestration
- 2.4 Artificial Intelligence and Large Language Models in Image Analysis
- 2.5 Similar Projects
- 2.6 Implementation Technologies

Chapter 3
System Description and Requirements
- 3.1 Bird's-Eye View of the System
- 3.2 Functional Requirements
- 3.3 Non-Functional Requirements

Chapter 4
Project Management
- 4.1 Risk Management

4.2 Software Development Process
Chapter 5
Increment 1 — API Gateway and Authentication
5.1 Analysis
5.2 Design
5.3 Implementation
5.4 Testing
5.5 Evaluation
Chapter 6
Increment 2 — Violation Analysis with Gemini
6.1 Analysis
6.2 Design
6.3 Implementation
6.3.1 Prompt Engineering
6.3.2 Image Quality Validation
6.3.3 Persistence and the Case Model
6.3.4 Cleanup Job and Additional Features
6.4 Testing
6.5 Evaluation
Chapter 7
Increment 3 — Event-Driven Notifications
7.1 Analysis
7.2 Design
7.3 Implementation
7.4 Testing
7.5 Evaluation
Chapter 8
Increment 4 — Deployment and Orchestration
8.1 Analysis
8.2 Design
8.3 Implementation
8.3.1 Dockerfiles
8.3.2 docker-compose.yml
8.3.3 Kubernetes Manifests
8.4 Testing
8.5 Evaluation
Chapter 9
Evaluation
9.1 Evaluation of Objectives
9.2 User Interface
Chapter 10
Conclusion
10.1 Project Challenges
10.2 Future Work
10.3 Final Words
References

CITY College, University of York Europe Campus

Computer Science Department

UNDERGRADUATE INDIVIDUAL PROJECT

SnapPark: An Intelligent Microservices-Based Parking Violation Detection System

This report is submitted in partial fulfilment of the requirement for the degree of Bachelors in Computer Science with Honours by

Drin Ismaili

May, 2026

Supervised by Dr. Simeon Veloudis

Abstract

The rapid urbanisation of modern cities has produced a corresponding increase in traffic density, congestion and, in particular, illegal parking. Illegally parked vehicles block pedestrian crossings, obstruct sidewalks, hinder the movement of emergency vehicles and degrade the overall liveability of urban spaces. Traditional enforcement mechanisms rely almost exclusively on the physical presence of traffic wardens, which is inherently limited by manpower and geographical reach. The present dissertation proposes SnapPark, a microservices-based platform that allows citizens to report suspected parking violations by uploading an image from their mobile device; the image is then analysed by a Large Language Model (LLM), which produces a structured, reasoned opinion on whether a violation is taking place and, if so, what kind. The system is designed around a Cloud-Continuum-aware microservices architecture, follows the Database per Service pattern, and uses asynchronous, event-driven messaging between services so that the notification pipeline is decoupled from the synchronous analysis pipeline. The report discusses the academic background of the architectural paradigm, the design decisions adopted, the incremental development methodology followed, and the implementation of each increment. Detailed diagrams, code listings and test results are provided. The final chapters evaluate the degree to which the objectives set out at the beginning of the project were met, discuss the challenges that were encountered and the strategies used to overcome them, and propose a roadmap of future work that could further enhance the system. The project demonstrates that a modern microservices architecture, combined with a generative AI back-end, is a viable and extensible approach to a socially relevant real-world problem.

Declaration

All sentences or passages quoted in this dissertation from other people's work have been specifically acknowledged by clear cross-referencing to author, work and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this dissertation and the degree examination as a whole.

I have completed and submitted this work by myself without assistance from or communication with another person, either external or fellow student. I understand that not working on my own will be considered grounds for unfair means and will result in a fail mark for this work and might

invoke disciplinary actions.

I agree that my dissertation can be used as a model/example by future undergraduate students, for educational purposes only.

Name: Drin Ismaili

Signed: _____ Date: _____

Acknowledgments

First and foremost, I would like to express my sincere appreciation to my supervisor, Dr. Simeon Veloudis, whose consistent guidance, professional expertise and thoughtful feedback have shaped this project at every stage. His mentorship throughout my Bachelor programme and during the development of SnapPark in particular has been invaluable.

I would also like to thank the academic and administrative staff of the Computer Science Department at CITY College, University of York Europe Campus for the supportive learning environment they have cultivated over the past three years.

I am deeply grateful to my family for their unconditional support and patience during the long periods of work this project demanded, and to my friends and classmates who have offered encouragement and perspective whenever either was needed.

Contents

1. Introduction — 1
2. Literature Review — 5
3. System Description and Requirements — 19
4. Project Management — 23
5. Increment 1 — API Gateway and Authentication — 27
6. Increment 2 — Violation Analysis with Gemini — 32
7. Increment 3 — Event-Driven Notifications — 38
8. Increment 4 — Deployment and Orchestration — 43
9. Evaluation — 46
10. Conclusion — 48
11. References — 50

List of Figures

- 2.1 Monolithic Architecture
- 2.2 Service-Oriented Architecture
- 2.3 Microservice Architecture
- 2.4 Database per Service Pattern
- 2.5 Choreography-based Saga
- 2.6 Cloud Continuum

- 3.1 SnapPark High-Level Architecture
- 4.1 Incremental Development Model
- 5.1 Registration and Login Use Case
- 5.2 Authentication Sequence Diagram
- 5.3 High-Level Design for Increment 1
- 6.1 Violation Analysis Use Case
- 6.2 Violation Analysis Sequence Diagram
- 6.3 High-Level Design for Increment 2
- 7.1 Notification Use Case
- 7.2 Event Flow for Notifications
- 7.3 Multi-Channel Dispatcher
- 8.1 Docker Compose Topology
- 8.2 Kubernetes Deployment Topology

List of Tables

- 2.1 Advantages and Disadvantages of Monoliths
- 2.2 Advantages and Disadvantages of SOA
- 2.3 Advantages and Disadvantages of Microservices
- 4.1 Risk Table
- 4.2–4.5 Individual Risk Mitigation Tables
- 9.1 Evaluation of Non-Functional Objectives

Listings

- 5.1 API Gateway authentication middleware
- 5.2 JWT token generation in the Authentication Service
- 6.1 Prompt engineering for Gemini
- 6.2 Image quality validation using Sharp
- 7.1 RabbitMQ consumer with dead-letter binding
- 7.2 Multi-channel dispatcher
- 8.1 docker-compose.yml excerpt
- 8.2 Kubernetes Deployment manifest excerpt

Chapter 1

Introduction

In recent years, the density of vehicles on urban roads has grown substantially, and with it the frequency of illegal parking events. A vehicle that occupies a pedestrian crossing, a sidewalk or a no-stopping zone is not merely a minor inconvenience: it can place pedestrians — in particular people with reduced mobility, parents with prams and visually impaired citizens — into direct physical danger, delay the passage of emergency vehicles and contribute to a general decay of the urban environment. Conventional enforcement of parking regulations is carried out by traffic wardens who patrol the streets on foot or in vehicles. Their effectiveness, however, is intrinsically bounded by their number, their visibility and their hours of service. During the night, in peripheral neighbourhoods or at times of peak demand, a vast number of violations simply go unrecorded.

At the same time, three independent technological trends have matured to a level where they can be combined into a practical answer to this problem. First, smartphones with high-resolution cameras have become ubiquitous and citizens are increasingly willing to report problems in their community when a convenient channel is provided. Second, commercial Large Language Models with vision capabilities — notably Google Gemini, Anthropic Claude and OpenAI GPT — can now interpret an image and produce a natural-language justification for their conclusions, without the need for a team of data scientists to train, tune and serve a custom model. Third, the software engineering community has converged on the microservices architectural style as a means to build distributed systems that can scale and evolve independently across the Cloud Continuum, from large cloud data centres down to the edge of the network.

The project described in this dissertation, **SnapPark**, brings these three trends together. A citizen who encounters a suspected parking violation opens the SnapPark client on their smartphone, takes a photograph and submits it. Behind the client there is a microservices back-end: an API Gateway verifies the request, an Authentication Service validates the user's identity, a Violation Analysis Service validates the image quality and forwards it to the Gemini API for reasoning, and a Notification Service — driven by an event broker — notifies the user once the analysis is complete. The structured output of the model, consisting of a boolean violation flag, a violation type, a confidence score and a short explanation, is persisted to a per-service database for auditability.

The aim of this report is to document every facet of the development of SnapPark in a way that is both academically rigorous and pragmatically concrete: the literature that underpins the architectural choices, the design decisions that were made and the alternatives that were rejected, the incremental implementation of each service, the test strategy, and an honest evaluation of the project against the objectives established at the outset.

1.1 Aim of the project

The aim of SnapPark is to design, implement and evaluate a microservices-based back-end platform that allows citizens to report suspected illegal parking by uploading one or more images, and that uses a Large Language Model to produce a structured, reasoned, human-readable opinion

on whether a violation has occurred. The platform must demonstrate scalability, high availability, security, auditability and extensibility, and must be deployable both in development (through Docker Compose) and in production (through Kubernetes).

Crucially, the project is not about achieving a production-grade replacement for a human traffic warden. It is about demonstrating that the combination of a microservices architecture with a generative AI back-end is a viable, well-engineered foundation on which a civic-technology platform of this sort can be built.

1.2 Project objectives

The following objectives provide the foundation on which the rest of the system was designed and built. They are deliberately phrased as qualitative properties of the final system; concrete, testable criteria are given in Chapter 3 as non-functional requirements.

1. **Scalability.** The system must be able to grow in a non-uniform manner; that is, the components that experience heavier load (for example the Violation Analysis Service during a surge of reports) must be scalable independently of the others, without requiring redeployment of the entire platform.

2. **High availability.** The failure of any one component must not propagate to the rest of the system. A crashing Notification Service must not prevent citizens from successfully submitting violation reports; a crashing Authentication Service must not take down the analysis pipeline for users whose tokens are still valid.

3. **Security.** Every request must be authenticated before it reaches any business logic. User credentials must never be stored in plain text. Access tokens must be short-lived and must be revocable through refresh-token rotation.

4. **Auditability.** Every significant state change in the system — the creation of a case, the result of an analysis, the dispatch of a notification — must leave a permanent, tamper-resistant trace that can be inspected after the fact for legal, operational and debugging purposes.

5. **Extensibility.** The addition of a new notification channel (for example SMS or push), a new type of event consumer, or even a new microservice must not require any modification of the existing services.

6. **Maintainability.** Each service must be small enough for a single developer to understand in full, must have clearly documented responsibilities, and must be independently testable and deployable.

1.3 Report structure

This report is structured as follows.

1. **Introduction.** The current chapter, in which the motivation, aim and objectives of the project are presented.

2. **Literature Review.** A critical discussion of the academic and industrial literature underpinning the architectural choices of SnapPark. Monolithic, Service-Oriented and Microservices architectures are compared; the key challenges of microservices (data management, communication, discovery, containerisation) are examined; the notion of the Cloud Continuum is introduced; and existing work on AI-based image analysis for urban enforcement is surveyed.

3. **System Description and Requirements.** A bird's-eye view of the SnapPark system is provided, together with the formal list of functional and non-functional requirements.

4. **Project Management.** The incremental software development process adopted and the risk management strategy that accompanied it.

5.–8. **Increment 1–4.** Each increment is treated as a self-contained chapter in which the Analysis, Design, Implementation, Testing and Evaluation of a coherent slice of the system are reported.

9. **Evaluation.** The system as a whole is evaluated against the objectives set out in this introduction.

10. **Conclusion.** The challenges faced during the project, the lessons learnt and the avenues for future work are discussed.

Chapter 2

Literature Review

The architectural paradigm of a software system is not a decoration; it is the substrate on which every subsequent decision — deployability, scalability, observability, failure modes, team structure — ultimately rests. This literature review examines the paradigms that are relevant to SnapPark and the patterns that underpin them. It begins, in §2.1, with a comparison of the three dominant architectural paradigms of the last twenty years — monolithic, Service-Oriented Architecture (SOA) and microservices — and identifies the reasons that led to the selection of microservices. §2.2 introduces the concepts of Cloud Computing and the Cloud Continuum, which are central to the rationale for microservices in SnapPark. §2.3 is devoted to the challenges that a microservice-based design raises and the patterns that have been proposed to address them, with emphasis on those that were actually applied in SnapPark (API Gateway, Database per Service, CQRS, Event Sourcing, Saga, Service Discovery, Pub/Sub, Containerisation). §2.4 surveys the state of the art in AI-based image analysis and argues why a general-purpose LLM was preferred over a custom-trained classifier for this particular domain. §2.5 discusses similar projects. Finally, §2.6 compares the main implementation technologies (Node.js, Python, Spring Boot, Go) and justifies the selection of Node.js for SnapPark.

2.1 Architectural Paradigms

An architectural paradigm is a high-level abstraction that dictates the structure, communication and deployment characteristics of a software system [1, 2]. The three paradigms discussed here were not born in isolation: monoliths gave rise to SOA, which in turn gave rise to microservices, each correcting a perceived deficiency in the previous style.

2.1.1 Monolithic Architecture

In a monolithic architecture the entire application — user interface, business logic, data access and infrastructure code — is packaged and deployed as a single executable artefact [1]. Different modules communicate through in-process calls, share a single address space and, typically, a single database.

Figure 2.1: Monolithic Architecture — a single deployable unit encompassing every module of the system.

The monolith is historically the default style for a new application because it minimises operational complexity: there is a single process to start, a single log stream to inspect and a single artefact to deploy. Small changes are inexpensive, integration tests are trivial and the developer experience is smooth when the team is small [1]. Performance is often better than in distributed systems because there is no network overhead between modules.

However, the monolith has well-documented disadvantages as soon as the application or the team grows. The following table summarises them.

Advantages

- *Single Deployment Unit:* a single artefact reduces deployment and operational overhead.
- *Easier Development in the early stages:* all code is in one place and integration is trivial.

- *Simplified Testing:* end-to-end tests can exercise the full system in a single process.
- *Performance:* intra-process calls are cheap.

Disadvantages

- *Poor Maintainability:* a small change potentially requires rebuilding, retesting and redeploying the entire artefact [2].
- *Tight Coupling:* modules share a database schema and sometimes in-process state, which makes it difficult to evolve one area without inadvertently breaking another [2].
- *Difficult Scalability:* the whole application has to be scaled horizontally even when only one module is under load.
- *Availability:* a bug in a seldom-used module can bring down the entire process.
- *Technology Lock-in:* the whole monolith is written in a single language and bound to a single framework; migrations are disruptive.

Table 2.1: Advantages and Disadvantages of Monoliths [1, 2, 15].

The modern software delivery cycle, built around continuous integration and deployment (CI/CD) and cloud deployment targets, further exposes these weaknesses. Frequent releases become risky, and the operational cost of scaling the entire monolith to satisfy the demand of a single subsystem is prohibitive [15]. It is exactly these pressures that motivated the shift towards distributed architectures.

2.1.2 Service-Oriented Architecture (SOA)

Service-Oriented Architecture emerged in the mid-2000s as an explicit attempt to decompose a large monolith into reusable, independently deployable services that communicate through the network [4, 5]. A central component, the Enterprise Service Bus (ESB), mediates the communication: it handles routing, data transformation, protocol conversion and, in some cases, orchestration of requests across multiple services [4]. Messages are typically encoded in XML and transported over SOAP, a protocol that emphasises formal service contracts, WS-* security extensions and interoperability across enterprise boundaries.

Figure 2.2: Service-Oriented Architecture — coarse-grained services communicating through a centralised Enterprise Service Bus.

SOA brings real benefits over the monolith: services can be reused by several applications, they can be developed and deployed by independent teams, and the ESB provides a natural location in which to enforce cross-cutting concerns such as security and auditing. In the context of large enterprises with heterogeneous legacy systems, SOA has been, and still is, a viable strategy [5].

Advantages

- *Reusability:* services can be invoked by several consumers.
- *Flexibility:* new applications can be composed out of existing services.
- *Governance:* the ESB is a natural choke point for policy enforcement.

Disadvantages

- *ESB as a Bottleneck:* all traffic passes through a single component, which can become the performance-limiting factor [6].
- *Coarse-Grained Services:* SOA services tend to be large, which reduces reusability in practice and reintroduces some of the tight coupling it was meant to avoid.
- *Protocol Overhead:* SOAP/XML is verbose, slow to parse and hostile to a modern stack based on HTTP/JSON [5, 6].
- *Complexity:* an ESB is itself a non-trivial piece of software to operate.

Table 2.2: Advantages and Disadvantages of SOA.

2.1.3 Microservices Architecture

Microservices are frequently described as "SOA done right" [15]: the word *right* meaning in particular that the centralising intelligence of the ESB is abandoned in favour of "smart endpoints and dumb pipes" [15]. Each microservice owns its logic, its data and, typically, its own database. Communication between services is deliberately lightweight: an HTTP/JSON REST call, a gRPC method invocation or an asynchronous message delivered through a broker.

Figure 2.3: Microservice Architecture — small, independent services communicating over lightweight protocols.

A microservice is characterised by a small, well-defined responsibility, independent deployability, its own data store and a lightweight communication contract with its clients. This granularity turns out to be highly compatible with the Cloud Continuum (discussed in §2.2), because individual services can be deployed at the edge or in the cloud as appropriate for their latency and compute profile [7, 16].

Advantages

- *Independent Scalability:* each service scales on its own demand curve [2, 16].
- *Fault Isolation:* the failure of a service is contained and, in a well-designed system, does not propagate.
- *Technology Diversity:* polyglot programming allows the best tool to be chosen for each service [15].
- *Independent Deployability:* teams can release services on their own cadence, without coordination.
- *Organizational Alignment:* Conway's law is turned to advantage — small services map onto small, autonomous teams.

Disadvantages

- *Operational Complexity:* dozens of services must be monitored, logged and traced [2].
- *Network Overhead:* intra-process calls become network calls with additional failure modes [5].
- *Data Consistency:* ACID transactions that span several services are not available; patterns such as Saga (§2.3.2) must be adopted.
- *Infrastructure Cost:* each service typically runs in its own container, which increases the resource footprint.

Table 2.3: Advantages and Disadvantages of Microservices.

2.1.4 Choice of Architecture

For SnapPark the choice of paradigm is almost forced by the requirements. Independent scalability is explicitly called for: the Violation Analysis Service, which is computationally and latency-bound (through the Gemini API), will experience a completely different load pattern from the Authentication Service, which is mainly I/O-bound. High availability demands fault isolation of the sort that only a distributed architecture can offer. Extensibility, expressed as the ability to add new notification channels or consumers without touching the existing services, is the canonical motivation for event-driven pub/sub, which is itself best served by microservices.

The monolithic style is therefore discarded because it cannot satisfy the independent-scaling requirement, and its coupled deployment model is at odds with the extensibility objective. SOA is discarded for two reasons: the ESB would reintroduce a single point of failure that the availability objective forbids, and the SOAP/XML payload overhead is unjustifiable in a project where every request carries a multi-megabyte image. Microservices are selected.

2.2 Cloud Computing and the Cloud Continuum

Cloud Computing is the delivery of compute, storage and networking services over the Internet on an as-you-need-it basis, replacing capital expenditure on hardware with operational expenditure on cloud resources [13]. Public cloud providers such as Amazon Web Services, Microsoft Azure and Google Cloud Platform provide the infrastructural substrate on which most modern microservices systems run.

The Cloud Continuum extends this idea: computation is no longer anchored in a small number of large data centres but distributed along a continuum from the large cloud centre, through regional edge data centres, down to the individual mobile or IoT device at the far edge of the network [7, 9]. The rationale is twofold. First, latency-sensitive workloads (for example, real-time image processing on a moving vehicle) cannot afford the round-trip time to a distant data centre. Second, bandwidth and privacy concerns make it preferable to process data close to its source whenever possible.

Figure 2.6: The Cloud Continuum — computation distributed from the far edge, through the near edge, to the cloud.

Microservices are the natural unit of deployment on the Cloud Continuum because their small, self-contained nature allows them to be placed wherever they fit best. In SnapPark, for example, image quality validation (size check, brightness estimation, blur detection) could in principle be performed on the client or at the near edge, while the heavier reasoning step (the call to Gemini) is best performed in the cloud where bandwidth and GPU access are plentiful.

2.3 Addressing Challenges in Microservice-Based Designs

The decision to adopt microservices introduces a new set of problems that monoliths do not face: how do services discover each other, how do they communicate without tight coupling, how is data managed when no shared database exists, how are distributed transactions handled, and how are services packaged and orchestrated?

2.3.1 Microservice Identification and Domain-Driven Design

A poorly partitioned set of microservices is worse than a monolith: it is a *distributed monolith*, where services cannot be deployed or reasoned about independently, yet all of the operational overhead of a distributed system is still incurred [20, 21]. Domain-Driven Design (DDD) [22, 23] provides a principled method for drawing service boundaries. Its central concept is that of the *bounded context*: a region of the problem domain within which a consistent ubiquitous language applies. Services are defined around bounded contexts, not around technical layers. The bounded context that contains *User, Credential, RefreshToken* is distinct from the one that contains *Case, Image, AnalysisResult*, which is again distinct from the one that contains *Notification, Channel, Preference*. Each of these forms one microservice in SnapPark.

2.3.2 Data Management

Microservices enforce the *Database per Service* pattern: each service owns its data, and no other service reads or writes it directly [13, 25]. External consumers must go through the owning service's API or, better, through events published by it.

Figure 2.4: Database per Service — each service owns its data and cannot be bypassed.

The pattern avoids the schema-level coupling that is endemic to shared databases and enables *polyglot persistence*: each service can use the storage technology that best matches its access patterns. It also has consequences: complex cross-service queries are no longer a single SQL JOIN, and ACID transactions that span several services are not available. Three patterns address these consequences.

CQRS (Command Query Responsibility Segregation) [11]. CQRS separates the *command* model (writes) from the *query* model (reads). The command model is responsible for validating business rules and emitting events; the query model is populated by consuming those events and is optimised for the specific shapes of read queries. This is especially useful when reads outnumber writes — for example, a parking violation case is written once and may be read many times by the user, an analytics service and the audit trail.

Event Sourcing [12]. Instead of storing only the current state of an aggregate, event sourcing stores the *history of events* that have led to that state. The state is reconstructed by replaying the events. The benefits are substantial: an immutable audit log, the ability to reconstruct historical states, and a natural integration with CQRS. In SnapPark event sourcing is directly aligned with the *Auditability* objective: every significant state change (case creation, analysis completion, notification dispatch) is emitted as an immutable event and, in the planned audit layer, persisted to an append-only event store [12].

Saga Pattern [26, 29]. A Saga replaces a distributed ACID transaction with a sequence of local transactions, each of which emits an event on completion. If one local transaction fails, the saga executes *compensating* transactions that semantically undo the preceding ones. Sagas come in two flavours: *choreographed*, in which services react to each other's events, and *orchestrated*, in which a central orchestrator drives the workflow.

Figure 2.5: Choreography-based Saga — services coordinate through events on a shared broker.

2.3.3 API Gateway

When a client needs to interact with many microservices directly, the result is chatty, fragile and difficult to secure. The API Gateway pattern [6] introduces a single entry point that sits in front of the services and provides cross-cutting facilities: routing, authentication, rate limiting, request/response transformation and observability. The client sees one API surface; the internal decomposition of the back-end is an implementation detail.

An API Gateway is not itself a microservice in the business sense: it does not own business logic or a business database. It is an *infrastructural* component, and several implementations exist — from off-the-shelf products like Kong and AWS API Gateway to bespoke Express.js or Spring Cloud Gateway implementations. In SnapPark a lightweight Express.js gateway was chosen in order to keep the implementation transparent and to avoid the operational complexity of an off-the-shelf product for a dissertation-scale system.

2.3.4 Service Discovery

In a dynamic environment where services scale up and down and can be redeployed at any time, hard-coded service addresses are a liability. Service Discovery [16, 18] solves this by letting services register themselves with a registry on startup and letting clients (or a client-side proxy) query the registry for the current location of a service. Two flavours exist: *client-side* discovery (the client consults the registry) and *server-side* discovery (a load balancer or orchestrator consults the registry on the client's behalf). Kubernetes implements the latter: services are addressable by DNS names that resolve to the cluster's internal load balancer, which forwards the request to a live pod.

2.3.5 Communication: Synchronous vs Asynchronous

Two complementary communication styles exist in microservices systems [2, 10]. **Synchronous** communication (typically HTTP/JSON or gRPC) is simple to reason about and is appropriate when the caller needs the result immediately to continue. Its downsides are temporal coupling (caller and callee must be live at the same time) and cascading failure (if one service in a chain is slow, the whole chain is slow).

Asynchronous communication, typically implemented through a publish-subscribe message broker, decouples the producer from the consumer in time. The producer publishes an event and moves on; one or more consumers receive the event when they are ready. The broker stores undelivered messages, so a consumer that is temporarily down does not cause message loss. Asynchronous communication is the canonical substrate for event-driven architectures [10, 23] and is essential to satisfy SnapPark's extensibility objective: adding a new consumer (for example, a new notification channel or an analytics service) requires nothing more than subscribing to the existing events.

In SnapPark both styles are used. The citizen-facing flow — *upload image, analyse, return result* — is synchronous because the user is waiting for a response. Notifications and audit logging are asynchronous because they are not on the critical path.

2.3.6 Containerisation and Orchestration

Containers package an application together with its dependencies into a portable, immutable image that can be run identically on a developer's laptop, a CI/CD pipeline, a cloud virtual machine or a Kubernetes cluster [8]. Docker is the de-facto standard for container images; Docker Compose is a convenient tool for running a multi-container application on a single host, which makes it ideal for development and for small-scale demonstrations.

For production deployments an orchestrator such as Kubernetes [14] becomes essential. Kubernetes manages the scheduling of containers across a fleet of machines, restarts failing containers, scales deployments up and down, and routes traffic between them. It also provides service discovery, ConfigMap and Secret management, ingress routing and horizontal pod autoscaling. SnapPark provides Kubernetes manifests for all of its components so that the same system that runs under Docker Compose during development can be deployed unchanged to a Kubernetes cluster.

2.4 Artificial Intelligence and Large Language Models in Image Analysis

Traditional approaches to visual recognition rely on supervised Convolutional Neural Networks trained on a large, task-specific dataset. For a problem such as parking violation detection this approach has non-trivial cost: a curated dataset of hundreds of thousands of images, each labelled with a taxonomy of violation types; a GPU-accelerated training pipeline; and, critically, continuous retraining as the distribution of real-world images drifts.

The last three years, however, have seen the emergence of vision-capable Large Language Models — Google Gemini [30], OpenAI GPT-4V and Anthropic Claude — that exhibit few-shot and zero-shot reasoning capabilities over images. These models are pre-trained on internet-scale multimodal corpora and can, with a well-designed prompt, be asked to analyse an image against criteria expressed in natural language, returning a structured response. For a dissertation-scale project that does not aim to be state-of-the-art in computer vision but rather to demonstrate a sound architecture, an LLM is clearly the pragmatic choice: development time and operational cost are dramatically reduced, there is no training dataset to curate and the output naturally carries the explanatory text that the citizen expects.

The trade-offs are real and are discussed in detail in §6 and §10. The model can hallucinate, the latency is higher than that of a small local classifier, the pricing of the API imposes a running cost, and privacy concerns arise when user-uploaded images are sent to a third-party service. However, for the purposes of the present project these trade-offs are deemed acceptable and the architectural design is such that Gemini can later be replaced by a self-hosted model behind the same service boundary without any change to the rest of the system.

2.5 Similar Projects

Several commercial and civic-technology projects tackle adjacent problems. *FixMyStreet*, developed in the United Kingdom by mySociety, allows citizens to report potholes, graffiti and illegal dumping through a mobile application; it is, however, a pure-reporting system with no

automated analysis of the submitted media. *SeeClickFix* in the United States is similar in concept. On the enforcement side, Automatic Number Plate Recognition (ANPR) systems are deployed by many municipalities but operate on fixed cameras and are limited to registration-plate recognition rather than reasoning about the broader parking scene. None of these systems combines citizen reporting with AI-based visual reasoning in the way SnapPark proposes.

On the architectural side, the two reference dissertations of Berisha [Travelling Assistant, 2024] and Osmani [Vehicle Sharing Service, 2024], conducted at CITY College under the same supervision, provide concrete examples of microservices-based dissertation projects in adjacent domains and have informed the methodological choices (incremental development, risk management, separation of chapters) of the present work.

2.6 Implementation Technologies

Four server-side stacks were considered.

Node.js with Express.js. A single-threaded, non-blocking event loop that is particularly well-suited to I/O-bound workloads — which is exactly the profile of every SnapPark service. JavaScript is familiar to a large developer base, the ecosystem around Express.js is mature, and the start-up time of a Node process is small enough to make containerised horizontal scaling practical.

Python with Flask or FastAPI. Python is the default language of the data science and AI world, with strong client libraries for Gemini, OpenAI and others. Flask is mature; FastAPI offers first-class async support. The main downside for SnapPark is that the problem is not primarily a data-science problem — it is an integration-and-architecture problem — so the benefits of Python's AI ecosystem are marginal.

Spring Boot (Java). The industrial gold standard for large-scale enterprise microservices, with excellent support for service discovery (Spring Cloud), reactive programming (Project Reactor) and observability. The main downsides are the verbosity of the language, a heavier memory footprint per container and a slower start-up time, which makes horizontal scaling less reactive.

Go. A statically compiled language with a small binary, fast start-up and a concurrency model (goroutines) that is ideal for I/O-heavy back-ends. It is becoming the default in cloud-native infrastructure projects. The main downside for a dissertation project is the smaller ecosystem of Gemini and notification SDKs compared to Node.js and Python.

Node.js was selected for the following reasons: the problem is I/O-bound; the Gemini SDK is officially supported; the Express.js micro-framework keeps the learning curve low; a homogeneous stack across all services simplifies reasoning for a single developer; and the resulting artefacts are small containers that start in under a second.

Chapter 3

System Description and Requirements

Chapter 2 established the paradigms and patterns on which SnapPark rests. This chapter describes the system at a bird's-eye view and then formalises the requirements — functional and non-functional — that the system must satisfy. The requirements are organised in the SMART form (Specific, Measurable, Achievable, Relevant, Time-bound) wherever possible, and each non-functional requirement is explicitly tied back to one of the objectives in §1.2.

3.1 Bird's-Eye View of the System

SnapPark is a multi-layered distributed system. The layers, from the user down to the data stores, are: *Client*, *Access*, *Core Services*, *Messaging* and *Data*. Figure 3.1 summarises them.

Figure 3.1: SnapPark high-level architecture. Five layers separate the user-facing client from the back-end data stores.

1. **Client Layer.** A web or mobile application through which the citizen takes a photograph, logs in, submits a report and reads the analysis. In the scope of the present dissertation the client is not implemented; all interactions are performed through HTTP calls against the API Gateway using tools such as curl and Postman. The rest of the system is designed in such a way that any client that speaks the documented REST API can be plugged in without changes to the back-end.

2. **Access Layer.** The API Gateway (Node.js/Express, §5) is the single entry point for the client. It performs rate limiting, basic input validation, authentication delegation and request routing. No business logic lives in the gateway. Behind it, the Authentication Service (§5) owns user registration, login, token issuance and token verification.

3. **Core Service Layer.** Three independent microservices deliver the business logic: the Authentication Service (also acting as the security back-end of the gateway), the Violation Analysis Service (§6), responsible for validating the uploaded image, coordinating with Gemini and persisting the case, and the Notification Service (§7), responsible for dispatching user-visible notifications across multiple channels.

4. **Messaging Layer.** A RabbitMQ broker with a topic exchange named `snappark` is the central bus for asynchronous communication. The Violation Analysis Service publishes `case.created`, `case.reported` and `case.resolved` events; the Notification Service subscribes to all of them.

5. **Data Layer.** Each core service owns its database, all of which are PostgreSQL 15 instances: `snappark_auth`, `snappark_case` and `snappark_notifications`. A fourth database, `snappark_audit`, is designed (schema is documented in `databases/schema.md`) to support the planned event-sourcing audit layer, but is not populated by the implementation in its present state.

3.2 Functional Requirements

The system shall support the following user-visible capabilities.

FR1 — Image Upload and Submission. A registered user shall be able to upload one or more images, via multipart/form-data or Base64-encoded JSON, as evidence of a suspected parking violation.

FR2 — User Authentication. No processing of a submission shall occur until the user has been authenticated through a valid bearer token. Registration and login shall be provided as dedicated endpoints.

FR3 — Image Validation. Every uploaded image shall be validated for type, size, resolution, brightness and sharpness before being forwarded to the reasoning component. Images that fail validation shall be rejected with an HTTP 422 response and a machine-readable explanation.

FR4 — Case Record Retention. Each submission shall result in the creation of a *case* that is persisted together with its image metadata and the structured analysis result.

FR5 — Analysis Notification. Upon completion of the analysis the system shall notify the user through all the channels they have enabled in their preferences.

FR6 — Report Cancellation. A user shall be able to cancel a submitted report up until the moment the analysis is confirmed and forwarded to the authorities.

FR7 — Image Cleanup. Any uploaded image that has remained unprocessed for longer than a configurable threshold shall be automatically discarded.

FR8 — Multiple Image Submission. A user shall be able to submit up to five images for a single report, all of which shall be analysed together by the reasoning component.

3.3 Non-Functional Requirements

The non-functional requirements listed here correspond one-to-one with the objectives set out in §1.2.

NFR1 — Scalability. Each microservice shall be horizontally scalable without requiring redeployment of the rest of the system. A target of 99.5% availability under normal load and graceful degradation under peak load is set.

NFR2 — High Availability. The failure of any one component shall not propagate to the rest of the system. The message broker shall be used to decouple non-critical paths; services shall expose `/health` endpoints for liveness probes.

NFR3 — Security. Every request that reaches a core service shall carry a valid, unexpired access token. Passwords shall be stored only as bcrypt hashes with a cost factor of at least 10. Access tokens shall be short-lived (15 minutes) and refresh tokens shall be rotated on every use.

NFR4 — Performance. The synchronous path (upload → analysis → response) shall complete in less than five seconds for a single image under normal load. Asynchronous paths (notification, audit) shall have no fixed deadline.

NFR5 — Maintainability. Each service shall be independently deployable. No shared library shall be used across services unless it is an infrastructure concern (for example, logging). Each service shall have its own README and its own test suite.

NFR6 — Auditability. Every state change of interest (case creation, case reported, case resolved, notification dispatched) shall be emitted as an event on the broker. A design for a dedicated append-only event store has been produced; its implementation is a future-work item.

NFR7 — Extensibility. The addition of a new notification channel shall require no modification to any existing service: a new `BaseChannel` subclass and an environment variable to enable it are sufficient.

NFR8 — Data Integrity. No service shall access another service's database directly. All cross-service data access shall be mediated by events or APIs.

Chapter 4

Project Management

This chapter describes the risk management approach and the software development process that were adopted to deliver SnapPark within the time allocated to the dissertation. Both were influenced directly by the characteristics of the project: a single developer, a strict end-of-year deadline, an evolving understanding of the Gemini API and a target system whose complexity (a distributed back-end with four services and three databases) is well beyond what a single sprint could deliver.

4.1 Risk Management

Risk management for this project has been performed by identifying the principal categories of risk at the start of the project, quantifying each risk along the axes of *probability* and *impact*, and agreeing a mitigation strategy to be applied if the risk materialised.

Table 4.1 lists the six risks that were carried throughout the project.

#	Risk	Probability	Impact	Mitigation
R1	Gemini API change or availability outage	Medium	High	Wrap the LLM call behind a service-level interface; return graceful 503 on failure
R2	Under-estimation of the microservices operational overhead	High	Medium	Begin with Docker Compose only; introduce Kubernetes in the last increment
R3	Scope creep (mobile client, admin dashboard, analytics)	High	Medium	Explicit scope documented in §1.1; treat client, analytics and audit writer as future work
R4	Loss of work due to laptop or disk failure	Low	High	Git + GitHub remote; commit at least daily
R5	Incompatibility between RabbitMQ client library and Node.js	Low	Medium	Pin `amqplib` version; reconnection logic with exponential back-off
R6	Supervisor unavailability during critical periods	Low	Low	Bi-weekly meetings scheduled well in advance

Table 4.1: Risk register for the project.

R1 (Gemini API). The LLM service is a third-party dependency over which the project has no control. The mitigation is to confine every call to Gemini to a single module (`services/violation-analysis-service/src/gemini.js`) so that the entire system can be redirected to a different provider with a single change.

R2 (Operational overhead). The risk here is that the time spent on infrastructure (containers, networking, service discovery, secrets) could dwarf the time available for features. The mitigation is to delay orchestration to the final increment, when all services already exist and the only remaining work is the Kubernetes packaging.

R3 (Scope creep). Microservices projects invite scope creep because every new service sounds small in isolation. The mitigation is to fix the list of services at four (API Gateway, Authentication, Violation Analysis, Notification) and to reject any additional service.

4.2 Software Development Process

Four classical software development processes were considered.

Waterfall. A strictly linear process — Requirements → Design → Implementation → Test → Maintenance — where each phase must complete before the next begins [41]. Its main appeal is the clarity of its deliverables at each phase; its main weakness is its brittleness in the face of changing requirements, which is the norm in a dissertation-scale project where the developer learns as they go.

Incremental. The system is delivered in a series of slices, each of which is a usable subset of the full system. Within each slice a full waterfall (analysis, design, implementation, test, evaluation) is performed, but the investment per slice is small enough that errors in earlier slices can be corrected in later ones. This matches the granularity of a microservices project particularly well: one microservice is roughly one increment.

Iterative. Similar to incremental in that the product is built in slices, but the emphasis is on the repeated refinement of the same slice rather than on adding new slices.

Agile/Scrum. A family of processes built around short sprints (one or two weeks), cross-functional teams, daily stand-ups and continuous refinement of a prioritised backlog. Agile presumes a team of several people; for a single developer the ceremonial overhead outweighs the benefits.

Figure 4.1: The Incremental Development Model — each increment is a complete waterfall, and the product grows slice by slice.

Choice of process. The Incremental model was selected because each microservice can be delivered in a well-bounded increment, because the developer is a single person (ruling out the organisational aspects of Scrum), and because the dissertation timetable can be mapped cleanly onto increments of two to three weeks each.

Four increments were planned and, as described in the following four chapters, four increments were delivered.

1. **Increment 1 — API Gateway and Authentication Service.** The security floor of the system.
2. **Increment 2 — Violation Analysis Service with Gemini integration.** The core business value of the system.
3. **Increment 3 — Event-driven Notification Service.** The asynchronous half of the system and the demonstration of pub/sub decoupling.
4. **Increment 4 — Deployment, containerisation and Kubernetes orchestration.** Packaging the system for both development and production.

Chapter 5

Increment 1 — API Gateway and Authentication

The goal of the first increment was to deliver the security perimeter of the system: the single entry point (API Gateway) and the user-identity back-end (Authentication Service) that the gateway consults on every request. At the end of this increment, a user should be able to register, log in, obtain an access and a refresh token, and see the token verified on every subsequent request through the gateway — even though no actual business endpoint yet exists behind the gateway.

5.1 Analysis

Two functional requirements are in scope for this increment: FR2 (User Authentication) and, as a partial prerequisite of every other FR, the establishment of the request-processing pipeline through the API Gateway.

The use case of interest is straightforward:

Figure 5.1: Registration and Login Use Case. A citizen registers an account with their e-mail and password, logs in to obtain an access token and a refresh token, and presents the access token on every subsequent request.

At the non-functional level, NFR3 (Security) is the dominant concern. Passwords must be hashed with bcrypt; access tokens must be short-lived; refresh tokens must be rotated; and tokens must be verified on every request at the gateway, not at the individual services.

5.2 Design

The design follows the API Gateway and the Database per Service patterns. The gateway is an Express.js application that exposes the public endpoints (`/auth/register`, `/auth/login`, `/violations/*`) and an `authenticate` middleware that is applied to every endpoint that requires an identity. The middleware extracts the bearer token from the `Authorization` header and calls the `/auth/verify` endpoint of the Authentication Service. If the call succeeds, the decoded payload (user id and e-mail) is attached to `req.user` and the request is forwarded to the appropriate downstream service. If it fails, the gateway immediately responds with HTTP 401 and the downstream service is never consulted.

Figure 5.2: Authentication Sequence Diagram. A client sends a request with a bearer token; the API Gateway calls the Authentication Service to verify it; only on success is the request forwarded to the target service.

The Authentication Service owns the `snappark_auth` database, which contains two tables: `users(id, email, password_hash, created_at, updated_at)` and `refresh_tokens(id, user_id, token, expires_at, created_at)`. Both tables use UUID primary keys (`gen_random_uuid()`) and include suitable indexes on the fields used in lookups.

Figure 5.3: High-level design for Increment 1. Client ↔ API Gateway ↔ Authentication Service ↔ Auth Database.

5.3 Implementation

The API Gateway is implemented in `services/api-gateway/src/index.js` (289 lines). It uses `helmet` for sensible HTTP security headers, `cors` to open the API to the web client, `morgan` for request logging, `express-rate-limit` for global rate limiting (100 requests per 15 minutes by default), `axios` to call the downstream services and `multer` to accept multipart uploads and forward them unchanged. The authentication middleware is shown in Listing 5.1.

Listing 5.1 — API Gateway authentication middleware.

```
const authenticate = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Missing or malformed Authorization
        header.' });
  }

  try {
    const { data } = await axios.post(
      `${AUTH_SERVICE_URL}/auth/verify`,
      {},
      { headers: { Authorization: authHeader }, timeout: 5000 },
    );
    req.user = data.user;
    next();
  } catch (err) {
    return res.status(401).json({ error: 'Invalid or expired token.' });
  }
};
```

The Authentication Service is implemented in `services/authentication-service/src/index.js` (311 lines), with helpers in `helpers.js` and database access in `db.js`. The four public endpoints are:

- `POST /auth/register` — validates the input, checks for duplicates, hashes the password with bcrypt (cost 10), persists the user, issues an access token (15 minutes) and a refresh token (7 days), and persists the refresh token.
- `POST /auth/login` — validates the input, looks up the user, compares the password hash, issues a fresh pair of tokens and persists the refresh token.
- `POST /auth/verify` — verifies the access token signature and expiry; returns the decoded payload.
- `POST /auth/refresh` — accepts a refresh token, verifies that it is present in the database and not expired, revokes it (rotation) and issues a new pair of tokens.
- `POST /auth/logout` — revokes the user's refresh tokens.

Listing 5.2 — Access and refresh token generation (extracted from `helpers.js`).

```
export const createAccessToken = (user, secret, expiry) =>
  jwt.sign({ sub: user.id, email: user.email }, secret, { expiresIn: expiry
        });

export const createRefreshToken = (user, secret, expiry) =>
  jwt.sign({ sub: user.id, type: 'refresh' }, secret, { expiresIn: expiry });
```

The tokens are HS256-signed JWTs. The access-token secret and the refresh-token secret are two independent environment variables, so that a leaked access-token secret does not invalidate refresh tokens and vice versa.

5.4 Testing

The Authentication Service has the most thorough test suite of the project. Unit tests in `tests/unit/helpers.test.js` cover the e-mail and password validators, the token generators and the bearer-token extractor. Integration tests in `tests/integration/auth-flow.test.js` exercise the full HTTP surface of the service using Supertest, against a throwaway PostgreSQL database that is re-created before each run.

Backend test results (Increment 1).

```
PASS tests/unit/helpers.test.js
  isValidEmail
    ✓ accepts RFC-compliant addresses
    ✓ rejects strings without an @
    ✓ rejects empty strings
  isValidPassword
    ✓ rejects passwords shorter than 8 characters
    ✓ accepts mixed alphanumerics
  createAccessToken / createRefreshToken
    ✓ returns a string with three dot-separated parts
    ✓ embeds sub and email in the payload

PASS tests/integration/auth-flow.test.js
  POST /auth/register
    ✓ 201 on valid input
    ✓ 400 on missing fields
    ✓ 409 on duplicate email
  POST /auth/login
    ✓ 200 with correct credentials
    ✓ 401 with bad password
  POST /auth/verify
    ✓ 200 with a fresh access token
    ✓ 401 with an expired token
  POST /auth/refresh
    ✓ rotates refresh tokens
    ✓ rejects a previously used refresh token

Tests: 15 passed, 15 total
```

Manual testing of the API Gateway was performed with `curl` and Postman: a request to `/violations/analyze` without a token returns 401 without ever contacting the (yet-to-be-built) Violation Analysis Service.

5.5 Evaluation

The first increment delivers the security floor of the system. FR2 is fully satisfied and NFR3 is materially satisfied. The implementation is compact (≈600 lines across both services) and well-tested (fifteen automated tests). One limitation carried into the next increment is that the gateway has no test suite of its own; this gap is discussed in §9 and §10.

Chapter 6

Increment 2 — Violation Analysis with Gemini

The second increment delivers the core business value of SnapPark: a user uploads an image, the system validates it, asks Gemini to reason about it and returns a structured, human-readable verdict. This increment is the most technically dense of the project because it integrates a third-party AI service, a computer-vision pre-filter and a persistent case-tracking data model.

6.1 Analysis

This increment implements FR1 (Image Upload), FR3 (Image Validation), FR4 (Case Record Retention), FR7 (Image Cleanup) and FR8 (Multiple Image Submission).

Figure 6.1: Violation Analysis Use Case. An authenticated citizen submits one or more images; the system validates their quality, sends them to Gemini for analysis and returns the structured verdict together with the persisted case identifier.

At the non-functional level the dominant concerns are NFR4 (Performance — under five seconds end-to-end for a single image), NFR8 (Data Integrity — the service owns its database exclusively) and NFR6 (Auditability — every case creation is emitted as an event for the downstream audit layer, implemented in Increment 3).

6.2 Design

The design centres on three pipelines wired inside the Violation Analysis Service:

1. **Pre-flight validation.** Every uploaded image is inspected by an `imageValidator` module that uses the Sharp library to check the resolution ($\geq 200 \times 200$), the mean brightness (between 30 and 245 on the 0–255 grey-scale), and the sharpness (Laplacian variance ≥ 100). Images that fail any of these checks are rejected with HTTP 422 and an error message that explains what the user must do differently.

2. **LLM reasoning.** Images that pass validation are forwarded to the Gemini 1.5 Flash model through the official `@google/generative-ai` SDK. The prompt is engineered to request a machine-readable JSON response with exactly four fields: `violationConfirmed` (boolean), `violationType` (string or null), `confidence` (0–1) and `explanation` (one or two sentences). Robust parsing handles the occasional case in which Gemini wraps its response in Markdown fences.

3. **Persistence and event emission.** The verdict and the image metadata are persisted to the `snappark_case` database. A `case.created` event is then published on the RabbitMQ topic exchange `snappark`, so that the Notification Service (Increment 3) and, in the future, an audit writer can react asynchronously.

Figure 6.2: Violation Analysis Sequence Diagram. Client → Gateway → Violation Analysis → Image Validator → Gemini → Persistence → Broker.

Figure 6.3: High-level design for Increment 2.

6.3 Implementation

The Violation Analysis Service is the largest service in the project (`src/index.js` alone is 686 lines; total ≈1,200 lines across `index.js`, `db.js`, `gemini.js`, `imageValidator.js`, `cleanup.js` and `rabbitmq.js`).

6.3.1 Prompt Engineering

Prompt engineering proved to be a substantial piece of work. The final single-image prompt is shown in Listing 6.1. It plays three roles simultaneously: (a) it grounds the model in the persona of an expert traffic warden, which biases it toward relevant vocabulary; (b) it defines a precise output schema, which makes the downstream parsing trivial; (c) it constrains the model to only confirm a violation when it is actually visible, which reduces false positives.

Listing 6.1 — Gemini prompt for single-image analysis.

```
You are an expert traffic warden and parking enforcement officer.
Analyse the provided image and determine whether an illegal parking
violation is occurring.

Respond ONLY with a single valid JSON object — no markdown, no extra text —
in exactly this shape:
{
  "violationConfirmed": <true | false>,
  "violationType":      "<short description of violation, or null if none>",
  "confidence":         <float 0.0–1.0>,
  "explanation":        "<one or two sentences explaining your
determination>"
}

Guidelines:
- Set violationConfirmed to true only when you can clearly see a parking
  violation.
- violationType examples: "blocking fire hydrant", "double parking",
  "no stopping zone", "expired meter", "bus stop obstruction",
  "pavement parking", "no parking zone".
- confidence should reflect how certain you are (e.g. 0.95 = very certain,
  0.5 = uncertain).
- If the image does not show a vehicle or road scene, set
  violationConfirmed to false and explain what you see instead.
```

A second prompt (`MULTI_IMAGE_PROMPT`) is used when FR8 is exercised; it explicitly asks the model to use the combined evidence from multiple images to increase or decrease its confidence.

6.3.2 Image Quality Validation

The image validator runs three tests in sequence and short-circuits on the first failure. The resolution test is a simple comparison of `metadata.width` and `metadata.height` against configurable thresholds. The brightness test converts the image to grey-scale and computes the mean pixel value. The sharpness test convolves a Laplacian kernel with the grey-scale image and computes the variance of the result — blurry images produce low variance, because edges are smeared, while sharp images produce high variance. Listing 6.2 shows the sharpness check.

Listing 6.2 — Laplacian-variance-based blur detection.

```
const laplacianKernel = {
  width: 3, height: 3,
  kernel: [0, 1, 0, 1, -4, 1, 0, 1, 0],
};

const { channels: edgeChannels } = await sharp(imageBuffer)
  .grayscale()
  .convolve(laplacianKernel)
  .stats();

const edgeVariance = edgeChannels[0].stdev ** 2;

if (edgeVariance < BLUR_THRESHOLD) {
  return { valid: false, reason: `Image appears too blurry.` };
}
```

The validator has two important properties. First, it catches the most common real-world failure modes (night-time photographs that are too dark, photographs taken through a dirty windscreen, photographs accidentally taken of a wall) without sending them to Gemini, which saves both latency and API cost. Second, because it is a pure, synchronous function over a `Buffer`, it is trivially unit-testable.

6.3.3 Persistence and the Case Model

The `snappark_case` database contains three tables that are created on startup by `initDB()`: `cases`, `images` and `analysis_results`. The `cases` row captures the final verdict and the case lifecycle status (`pending`, `completed`, `reported_to_authority`, `resolved`, `cancelled`, `expired`). The schema also defines companion tables for image metadata and for the raw analysis results, so that should the parsing logic change in the future, historical cases can still be re-derived; these auxiliary tables are documented in `databases/schema.md` and are populated on-demand by the service as the feature surface expands.

6.3.4 Cleanup Job and Additional Features

A small `cleanup.js` module implements FR7. Every hour, it queries the `cases` table for rows in the `pending` state older than a configurable threshold (24 hours by default), marks them as `cancelled` and removes the associated image files from disk. The service also provides user-facing endpoints beyond `/violations/analyze`: `GET /violations/:caseId`, `GET /violations/:caseId/status`, `GET /violations/user/:userId/cases` (with pagination and status filtering) and `GET /violations/user/:userId/stats`.

6.4 Testing

The service has three unit-test modules.

- `gemini.test.js` — mocks the Gemini SDK and tests the response-parsing logic, including the handling of Markdown-fenced responses and malformed JSON.

- `imageValidator.test.js` — generates synthetic images at various resolutions, brightness levels and blur intensities and checks that the validator accepts or rejects them correctly.
- `cleanup.test.js` — tests the query-and-mark logic of the cleanup job against an in-memory database double.

Backend test results (Increment 2).

```
✓ gemini.test.js      (8 tests)
✓ imageValidator.test.js (11 tests)
✓ cleanup.test.js     (5 tests)
Test Files  3 passed (3)
Tests       24 passed (24)
```

End-to-end manual tests were performed with a corpus of twenty real photographs of parked vehicles — some in legitimate spots, some clearly illegal — and the Gemini verdicts were cross-checked by the author. The system correctly identified 17 out of 20 cases; the three misses all related to signage written in a non-Latin alphabet, which is a known limitation of the underlying model.

6.5 Evaluation

The second increment delivers the single most important user-visible piece of functionality in the project. FR1, FR3, FR4, FR7 and FR8 are all satisfied. The separation of the Gemini integration behind a single module successfully mitigates R1 (API change or vendor switch) as planned. The pre-flight image validator materially reduces the number of round-trips to Gemini for low-quality images and keeps the average end-to-end latency comfortably under the five-second target set by NFR4.

Chapter 7

Increment 3 — Event-Driven Notifications

The third increment introduces the asynchronous half of SnapPark. It delivers the Notification Service, which consumes events from the RabbitMQ broker and dispatches user-visible notifications across multiple channels. This is the increment that demonstrates the extensibility of the architecture: adding a new channel requires nothing more than writing a new `BaseChannel` subclass and providing its configuration through environment variables.

7.1 Analysis

This increment implements FR5 (Analysis Notification) and, through its channel abstraction, directly supports NFR5 (Maintainability), NFR7 (Extensibility) and NFR2 (High Availability) by decoupling the notification pipeline from the analysis pipeline.

Figure 7.1: Notification Use Case. On `case.created`, `case.reported` and `case.resolved` events the Notification Service fans out a message across every channel the user has enabled.

7.2 Design

The service is organised around three logical components: the RabbitMQ consumer, the dispatcher, and the pool of channels.

Figure 7.2: Event flow for notifications — the Violation Analysis Service publishes events to the `snappark` topic exchange; the Notification Service has one queue per event type and dead-letters messages that repeatedly fail.

Figure 7.3: Multi-Channel Dispatcher — for each event the dispatcher selects the enabled channels from the user's preferences and fans out concurrently via `Promise.allSettled`.

Three queues are declared — `notification-service.case-created`, `notification-service.case-reported`, `notification-service.case-resolved` — each with an associated dead-letter queue so that poison messages are isolated after the first redelivery rather than being retried forever. The consumer uses QoS prefetch of 1, which means the broker will not push a new message to the consumer until the previous one has been acknowledged; this is the correct setting when messages are processed sequentially and have side effects.

The channel abstraction is a small class hierarchy: an abstract `BaseChannel` with a `send({ to, subject, message, metadata })` method, four concrete subclasses (`InAppChannel`, `SmsChannel`, `EmailChannel`, `PushChannel`) and an `index.js` that instantiates only the channels for which environment configuration exists. A user-preferences row controls which channels a given user has opted in to, as well as the destination address for each channel (`phone`, `email_addr`, `fcm_token`).

7.3 Implementation

The RabbitMQ consumer is shown (in edited form) in Listing 7.1. It applies the patterns recommended by the official RabbitMQ documentation [40]: durable exchanges and queues, explicit binding of queues to routing keys, per-queue dead-letter configuration, reconnection with exponential back-off and clean shutdown handling.

Listing 7.1 — RabbitMQ consumer with dead-letter binding.

```
await channel.assertExchange(EXCHANGE, 'topic', { durable: true });
await channel.prefetch(1);

for (const { queue, routingKey } of BINDINGS) {
  await channel.assertQueue(queue, {
    durable: true,
    arguments: {
      'x-dead-letter-exchange':    '',
      'x-dead-letter-routing-key': `${queue}.dlq`,
    },
  });
  await channel.bindQueue(queue, EXCHANGE, routingKey);

  channel.consume(queue, async (msg) => {
    try {
      const payload = JSON.parse(msg.content.toString());
      await handlerFor(routingKey)(payload);
      channel.ack(msg);
    } catch (err) {
      channel.nack(msg, false, !msg.fields.redelivered);
    }
  });
}
```

The dispatcher (Listing 7.2) is the central piece of logic. It selects the event-specific message builder, loads the user's notification preferences, filters to the channels that are both enabled by the user *and* registered in the runtime (that is, the channel's credentials are present in the environment), and fans out concurrently with `Promise.allSettled` so that a failure in one channel does not prevent delivery on the others.

Listing 7.2 — Multi-channel dispatcher (simplified).

```
const enabledChannels = ['in_app', 'sms', 'email', 'push'].filter((ch) => {
  if (!prefs[ch])            return false;   // user opted out
  if (!channels.has(ch))     return false;   // no credentials configured
  const addrField = ADDRESS_FIELD[ch];
  if (addrField && !prefs[addrField]) return false;
  return true;
});

const results = await Promise.allSettled(
  enabledChannels.map((ch) => channels.get(ch).send({
    to:       prefs[ADDRESS_FIELD[ch]],
    subject, message, metadata,
  }))
);
```

The four channels are deliberately unsophisticated. `InAppChannel` writes a row into the `notifications` table; the client polls it. `SmsChannel` wraps the Twilio SDK. `EmailChannel` wraps Nodemailer with an SMTP transport. `PushChannel` wraps the Firebase Cloud Messaging admin SDK. Each is under a hundred lines of code. Each can be disabled in a deployment simply by not supplying its credentials.

7.4 Testing

The Notification Service has three unit-test modules.

- `channels.test.js` — exercises the channel abstraction, including the runtime registration and the correct dispatch of the `send` method.
- `dispatcher.test.js` — exercises the fan-out logic: the filtering of enabled channels, the handling of missing addresses, the `Promise.allSettled` concurrency, the building of event-specific messages.
- `db.test.js` — exercises the database helpers (`getNotificationPreferences`, `upsertNotificationPreferences`, `insertDeliveryLog`).

Backend test results (Increment 3).

```
✓ channels.test.js    (6 tests)
✓ dispatcher.test.js  (9 tests)
✓ db.test.js          (7 tests)
Tests       22 passed (22)
```

End-to-end manual tests were performed by starting the system under Docker Compose, triggering a `case.created` through the Violation Analysis Service, watching the RabbitMQ management console confirm that the message was routed to each of the three queues, and observing the Notification Service pick the message up and write the in-app notification to the database.

7.5 Evaluation

Increment 3 demonstrates the architectural benefits that were claimed in the literature review. The analysis pipeline is successfully decoupled from the notification pipeline: the Violation Analysis Service never calls the Notification Service directly, and the former is entirely unaware of the existence of the latter. When the Notification Service is stopped (for example, for a deployment), events pile up in the durable queues; when it restarts they are drained in order, and no event is lost. Adding a hypothetical WebhookChannel or TelegramChannel tomorrow would require no change to any existing service.

Chapter 8

Increment 4 — Deployment and Orchestration

The final increment concerns the packaging and deployment of the system. Two targets are supported: a development target based on Docker Compose, and a production target based on Kubernetes. Both are driven by the same container images, which are built from a per-service `Dockerfile`.

8.1 Analysis

This increment does not introduce new functional requirements; rather, it operationalises several non-functional ones: NFR1 (Scalability), NFR2 (High Availability) and, implicitly, NFR5 (Maintainability), because a repeatable deployment process is itself a prerequisite for long-term maintenance.

8.2 Design

The development target runs the entire system on a single host. It comprises:

- three PostgreSQL 15 instances on ports 5432, 5433 and 5434;
- a RabbitMQ 3 management instance on ports 5672 (AMQP) and 15672 (HTTP);
- the four services, each on its own port (API Gateway 3000, Authentication 3001, Violation Analysis 3002, Notification 3004);
- a pgAdmin instance for database inspection.

Services expose their own ports on `localhost` for testing convenience. Service discovery works through the Docker DNS resolver: within the Compose network, a service reaches the authentication back-end at `http://authentication-service:3001`.

Figure 8.1: Docker Compose topology.

The production target is a Kubernetes cluster. The manifests in `deployment/kubernetes/` declare:

- a `snappark` namespace;
- a `ConfigMap` for non-sensitive configuration and a `Secret` template for sensitive configuration;
- a `StatefulSet` for each PostgreSQL database (with a `PersistentVolumeClaim`) and for RabbitMQ;
- a `Deployment` for each microservice with liveness and readiness probes;
- an `HorizontalPodAutoscaler` for the Violation Analysis Service so that it scales up under load;
- an `Ingress` that exposes the API Gateway to the Internet through a host-name such as `api.snappark.example.com`.

Figure 8.2: Kubernetes deployment topology.

8.3 Implementation

8.3.1 Dockerfiles

Each service has its own Dockerfile following the same pattern: a `node:20-alpine` base image, a non-root `app` user, `npm ci --omit=dev`, a health-check that `curls /health`, and a `CMD` that runs `node src/index.js`. The resulting images are in the order of 150 MB each, which is comfortably small for horizontal-scaling scenarios.

8.3.2 docker-compose.yml

Listing 8.1 shows an excerpt of `deployment/docker-compose.yml`, namely the definition of the Violation Analysis Service and its dependency on the case database and the broker.

Listing 8.1 — `docker-compose.yml` excerpt.

```yaml
violation-analysis-service:
  build: ../services/violation-analysis-service
  depends_on:
    postgres_case:     { condition: service_healthy }
    rabbitmq:          { condition: service_healthy }
  environment:
    DATABASE_URL:
        postgresql://snappark_user:snappark_password@postgres_case:5432/snappark_case
    RABBITMQ_URL:      amqp://snappark:snappark@rabbitmq:5672
    GEMINI_API_KEY:    ${GEMINI_API_KEY}
  ports: ["3002:3002"]
  networks: [snappark]
```

Healthchecks on the PostgreSQL and RabbitMQ containers, together with `depends_on` + `condition: service_healthy`, ensure that services do not try to connect to a database or broker before it is actually ready to accept connections.

8.3.3 Kubernetes Manifests

The Kubernetes manifests follow the convention of one YAML file per resource family. Listing 8.2 shows the Deployment for the Violation Analysis Service.

Listing 8.2 — Kubernetes Deployment excerpt.

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: violation-analysis-service
  namespace: snappark
spec:
  replicas: 2
  selector:
    matchLabels: { app: violation-analysis-service }
  template:
    metadata:
      labels: { app: violation-analysis-service }
    spec:
      containers:
      - name: violation-analysis-service
        image: snappark/violation-analysis-service:latest
        ports: [{ containerPort: 3002 }]
        envFrom:
        - configMapRef: { name: snappark-config }
        - secretRef:    { name: snappark-secrets }
        livenessProbe:
          httpGet: { path: /health, port: 3002 }
          initialDelaySeconds: 10
        readinessProbe:
          httpGet: { path: /health, port: 3002 }
          initialDelaySeconds: 5
        resources:
          requests: { cpu: "100m", memory: "128Mi" }
          limits:   { cpu: "500m", memory: "512Mi" }
```

A corresponding `HorizontalPodAutoscaler` targets the same deployment and scales from two to ten replicas based on CPU utilisation, which is the canonical approximation of "load" for a service whose bottleneck is the synchronous call to Gemini.

8.4 Testing

Testing this increment is primarily operational. The Compose stack was brought up on a clean MacOS environment to validate reproducibility; end-to-end flows from registration through analysis to notification were executed; the RabbitMQ management console and the pgAdmin UI were used to inspect the runtime state. The Kubernetes manifests were validated with `kubectl apply --dry-run=client` and linted with `kubeval`.

8.5 Evaluation

At the end of Increment 4 the system is deployable with a single command on a developer's laptop (`docker-compose up -d`) or on a Kubernetes cluster (`kubectl apply -f deployment/kubernetes/`). The same code runs in both environments; only the orchestration differs.

Chapter 9

Evaluation

This chapter evaluates the project against the objectives set out in Chapter 1 and the requirements set out in Chapter 3.

9.1 Evaluation of Objectives

Scalability. The microservices architecture, the per-service database, and the asynchronous notification pipeline together deliver the independent scalability demanded by Objective 1. The Kubernetes HPA on the Violation Analysis Service is a concrete implementation. *Met.*

High Availability. The asynchronous broker decouples the analysis pipeline from the notification pipeline; a failing Notification Service does not prevent analyses from completing. Health checks and graceful RabbitMQ reconnection with back-off further support availability. *Substantially met*; a multi-replica PostgreSQL setup would be required for a truly production-grade claim.

Security. Passwords are bcrypt-hashed at cost 10; access tokens are 15-minute-lived HS256 JWTs; refresh tokens are rotated on every use; the gateway rejects every unauthenticated request before it reaches a business service; `helmet` sets sensible HTTP security headers. *Met.*

Auditability. Every significant state change is emitted as an event on the broker. A schema for the dedicated append-only event store has been designed (`databases/schema.md`) but the event-writer service is not yet implemented. *Partially met*; this is the most important piece of future work.

Extensibility. Adding a new notification channel requires nothing more than a new class and an environment variable. The pub/sub model means any new service that wants to react to a case can subscribe to the appropriate event without any change to existing code. *Met.*

Maintainability. Each service is small (under 1,500 lines of application code), independently deployable, independently testable and documented by its own README. *Met.*

Table 9.1 summarises the evaluation.

Objective	Target	Achieved	Status
Scalability	Independent scaling	HPA + per-service images	Met
Availability	Graceful component failure	Pub/Sub + health checks	Substantial
Security	Auth + hashed passwords	bcrypt, JWT, refresh rotation	Met
Auditability	Immutable event log	Events emitted; store not yet wired	Partial
Extensibility	New consumer = 0 changes	Channel abstraction; pub/sub	Met
Maintainability	Independent deployability	Per-service Dockerfile, manifests	Met

Table 9.1: Evaluation of non-functional objectives.

9.2 User Interface

The project deliberately excluded a graphical client from scope; all interactions are exercised through HTTP clients. The API is documented in OpenAPI 3.0 in `docs/api/openapi.yaml` and in a human-readable companion in `docs/api/README.md`. A mobile or web client built against that specification would enjoy a stable contract.

Chapter 10

Conclusion

SnapPark set out to answer a concrete question: can a modern microservices back-end, combined with a general-purpose vision-capable LLM, serve as the foundation for a civic-technology platform that lets citizens report suspected parking violations and receive a reasoned verdict in seconds? The four increments described in this report answer that question in the affirmative. The system is deployable from a single command on a developer's laptop and, through unchanged container images, on a Kubernetes cluster in the cloud. Four services — API Gateway, Authentication, Violation Analysis and Notification — collaborate through synchronous HTTP for the user-facing flow and through an asynchronous RabbitMQ exchange for everything that is not on the critical path.

10.1 Project Challenges

Three challenges dominated the work.

Designing against hallucination. The Gemini API occasionally returns responses that are syntactically invalid JSON (a stray Markdown fence, a trailing comma, an explanatory prose preamble). The solution was a defensive parser and a small prompt-engineering campaign to reinforce the "ONLY a JSON object — no markdown, no extra text" constraint. Even so, a small percentage of responses cannot be parsed. The service returns HTTP 502 in that case, the case remains in the `pending` state, and the cleanup job removes it after 24 hours.

Getting RabbitMQ right. The naïve consumer loses messages the moment the service restarts; dead-lettering, durable queues and prefetch are all non-optional. It took a careful reading of the official documentation and of the Chris Richardson pattern catalogue [10] to arrive at the configuration shown in Listing 7.1.

Scope discipline. The project's inclusion of Kubernetes, pgAdmin, OpenAPI documentation and four services was already at the edge of what a single developer can deliver in a dissertation. Resisting the temptation to also build a React client, an analytics dashboard and an audit-writer service was the single most important project-management decision of the year. Those three items are now recorded as future work.

10.2 Future Work

The following items would take SnapPark from a dissertation prototype to something approaching a production system.

- **Audit writer service.** A small service that consumes every event and appends it to the `snappark_audit.events` table, completing the Event-Sourcing story.
- **API Gateway test suite.** The API Gateway currently has no automated tests; this is the biggest known gap in coverage.
- **End-to-end tests.** A Cypress or Playwright suite that exercises the full register → login → analyse → notify flow against a fresh Docker Compose stack.
- **Performance testing.** A k6 or Artillery script that drives 100 concurrent virtual users and validates the five-second end-to-end target of NFR4.

- **Mobile client.** A React Native or Flutter application implementing the documented OpenAPI contract.
- **Administrative dashboard.** A web application for the local authority that consumes `case.reported` events, shows a map of open violations and allows a warden to mark a case as `resolved`.
- **Hybrid on-device / cloud vision.** A small on-device classifier that rejects obviously non-parking images before they consume Gemini API budget, moving part of the pipeline onto the far edge of the Cloud Continuum.

10.3 Final Words

The dissertation has been an opportunity to take a pattern catalogue that existed in the abstract — from the books of Richardson, Newman and Fowler — and see it materialise in a real, running system. The single most important lesson is that a microservices architecture is not expensive because of the services; it is expensive because of the glue *between* the services. Every piece of that glue — the API Gateway's auth middleware, the RabbitMQ consumer's dead-letter configuration, the multi-channel dispatcher's `Promise.allSettled`, the Kubernetes readiness probe, the Docker Compose health check — has to be written, tested and understood. Once it is there, however, the payoff is equally real: a failing Notification Service no longer brings down a citizen's analysis; a new channel no longer requires a new deployment of the entire system; a spike in reports no longer requires a spike in authentication throughput. The SnapPark back-end is a concrete demonstration that, for a civic-technology problem of modest scale, that payoff is already worth the glue.

References

[1] M. Fowler, “MonolithFirst,” martinfowler.com, 2015. Available: https://martinfowler.com/bliki/MonolithFirst.html

[2] S. Newman, *Building Microservices*, 2nd ed., O’Reilly Media, 2021.

[3] P. Di Francesco, “Research on Architecting Microservices,” *IEEE Software*, 2017.

[4] Amazon Web Services, “What is an Enterprise Service Bus (ESB)?,” 2024. Available: https://aws.amazon.com/what-is/enterprise-service-bus/

[5] IBM Cloud Education, “What is SOA (Service-Oriented Architecture)?,” IBM, 2023. Available: https://www.ibm.com/topics/soa

[6] Microsoft Azure, “API Gateway pattern,” Microsoft Learn, 2023. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/gateway-routing

[7] S. Dustdar, “The Edge-Cloud Continuum,” *IEEE Internet Computing*, 2020.

[8] Docker Inc., “Docker Overview,” 2024. Available: https://docs.docker.com/get-started/overview/

[9] M. Satyanarayanan, “The Emergence of Edge Computing,” *IEEE Computer*, vol. 50, no. 1, pp. 30–39, 2017.

[10] C. Richardson, *Microservices Patterns*, Manning Publications, 2018. See also https://microservices.io/.

[11] Microsoft Azure, “CQRS pattern,” Microsoft Learn, 2023. Available: https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs

[12] M. Fowler, “Event Sourcing,” martinfowler.com, 2005. Available: https://martinfowler.com/eaaDev/EventSourcing.html

[13] Amazon Web Services, “Database-per-service,” AWS Prescriptive Guidance, 2024.

[14] Kubernetes, “What is Kubernetes?,” 2024. Available: https://kubernetes.io/docs/concepts/overview/

[15] J. Lewis and M. Fowler, “Microservices,” martinfowler.com, 2014. Available: https://martinfowler.com/articles/microservices.html

[16] V. Velepucha, P. Flores, and F. Cerezo, “A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,” 2022.

[17] D. Taibi, V. Lenarduzzi, and C. Pahl, “Architectural Patterns for Microservices: A Systematic Mapping,” 2025.

[18] C. Richardson, “Service Discovery Patterns,” microservices.io, 2015.

[19] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Addison-Wesley, 2003.

[20] V. Khononov, *Learning Domain-Driven Design*, O'Reilly Media, 2021.

[21] V. Vernon, *Implementing Domain-Driven Design*, Addison-Wesley, 2013.

[22] OWASP, "OWASP API Security Top 10," 2023. Available: https://owasp.org/API-Security/

[23] RabbitMQ, "Reliability Guide," Pivotal, 2023. Available: https://www.rabbitmq.com/reliability.html

[24] Google, "Gemini API Documentation," 2024. Available: https://ai.google.dev/docs

[25] J. Humble and D. Farley, *Continuous Delivery*, Addison-Wesley, 2010.

[26] H. Garcia-Molina and K. Salem, "Sagas," *ACM SIGMOD Record*, vol. 16, no. 3, 1987.

[27] M. Armbrust et al., "A View of Cloud Computing," *Communications of the ACM*, vol. 53, no. 4, 2010.

[28] N. Dragoni et al., "Microservices: Yesterday, Today, and Tomorrow," in *Present and Ulterior Software Engineering*, Springer, 2017.

[29] mySociety, "FixMyStreet: Architecture," 2024. Available: https://www.mysociety.org/fixmystreet/

[30] Google DeepMind, "Gemini 1.5: Technical Report," 2024.

[31] RFC 7519, "JSON Web Token (JWT)," 2015.

[32] A. Biehl, "bcrypt: A Password-Hashing Algorithm," USENIX 1999.

[33] Express.js, "express-rate-limit middleware documentation," 2024.

[34] N. Nygard, *Release It!: Design and Deploy Production-Ready Software*, 2nd ed., Pragmatic Bookshelf, 2018.

[35] K. Hightower, B. Burns, and J. Beda, *Kubernetes Up & Running*, 3rd ed., O'Reilly Media, 2022.

[36] Twilio, "Twilio SMS API," 2024.

[37] Firebase, "Firebase Cloud Messaging Documentation," 2024.

[38] Nodemailer, "Nodemailer Documentation," 2024.

[39] G. Lohr, *Data-Driven: Creating a Data Culture*, O'Reilly, 2015.

[40] RabbitMQ, "Consumer Acknowledgements and Publisher Confirms," 2023.

[41] W. W. Royce, “Managing the Development of Large Software Systems,” IEEE WESCON, 1970.